UNIVERSIDAD NACIONAL DE CAÑETE

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

TRABAJO DE INVESTIGACIÓN

PharmaVictoria: Un Enfoque Arquitectónico Basado en Patrones de Diseño para la Gestión Farmacéutica

CURSO:

Arquitectura de software

INTEGRANTES:

Puma Palomino Esteban Ricardo

Loza Yataco Jesús Ángel

Barrientos Atocsa Jean Paul

Solano Ayaucán, Augusto Salvador

Agüela Palomino, Oliver Francisco

DOCENTE:

Ramirez Pacheco Luis Enrique

San Vicente de Cañete, 2025, Perú

ÍNDICE GENERAL

I. Introducción 3
II. Objetivos 5
2.1. Objetivo General 5
2.2. Objetivos Específicos 5
III. Antecedentes bibliográficos 7
Antecedentes Nacionales: 7
Antecedentes Internacionales: 9
IV. Marco Teórico 11
4.1. Arquitectura de Software 11
4.2. Patrones de Diseño (GoF) 13
4.3. Modelado Arquitectónico 16
V. Desarrollo del proyecto 17
5.1. Descripción del Sistema 17
5.2. Requerimientos 18
5.3. Patrones de Diseños Aplicados 21
VI. Modelado Arquitectónico 23
6.1. Modelo C4 23
Nivel 1 – Contexto 23
Nivel 2 – Contenedores 26
Nivel 3 - Componentes (Backend) 29
Nivel 4 – Código 32
VII. Resultados Esperados 40
VIII. Conclusiones 42
IX. Bibliografía 43

I. Introducción

El presente proyecto tiene como propósito aplicar los principios de arquitectura de software y patrones de diseño en el estudio y modelado de un sistema de escritorio denominado PharmaVictoria, orientado a la gestión integral de una farmacia local ubicada en el distrito de Imperial, provincia de Cañete. La investigación toma como caso real la farmacia PharmaVictoria, que cuenta con dos sucursales y un equipo aproximado de tres empleados por sede, quienes gestionan sus operaciones mediante un software interno básico.

Actualmente, la empresa busca optimizar y modernizar su entorno informático para mejorar la gestión de ventas, inventario, facturación y reportes administrativos. Su entorno operativo es local y de escritorio, con posibilidad de sincronización o conexión en red. A partir de este escenario, el propósito se enfoca en diseñar una arquitectura de software sólida y escalable, que sirva como base para el desarrollo o modernización del sistema existente, fortaleciendo la organización técnica y la eficiencia de los procesos.

El sistema propuesto busca ser modular, seguro y escalable, priorizando la correcta separación de responsabilidades entre las capas de presentación, lógica de negocio y datos. Con este enfoque, se pretende asegurar la mantenibilidad del software, la reutilización del código y la posibilidad de incorporar nuevas funcionalidades en el futuro sin comprometer la estabilidad del sistema.

Para lograrlo, la investigación aplica patrones de diseño (GoF) ampliamente reconocidos, como Singleton, Facade, Command, Observer, Factory Method, Builder y Strategy, los cuales permiten estructurar un modelo arquitectónico ordenado, flexible y preparado para la evolución tecnológica. Asimismo, se emplea el modelo C4 como herramienta de representación arquitectónica, permitiendo documentar de forma clara los distintos niveles del sistema: contexto, contenedores, componentes y código; y las interacciones entre los actores internos y externos, incluyendo los servicios regulatorios de SUNAT y DIGEMID.

En conjunto, esta investigación busca demostrar cómo el uso adecuado de la arquitectura de software y los patrones de diseño puede contribuir de manera directa a la modernización tecnológica de una empresa farmacéutica local, promoviendo una gestión más eficiente, segura y adaptable a las necesidades del entorno de Cañete.

II. Objetivos

2.1. Objetivo General

Diseñar y modelar una arquitectura de software moderna, modular y escalable para la farmacia PharmaVictoria, ubicada en el distrito de Imperial, provincia de Cañete, aplicando principios de arquitectura de software y patrones de diseño (GoF). El estudio toma como referencia el entorno real de la farmacia para proponer una solución tecnológica que optimice la gestión de ventas, inventario, facturación y reportes administrativos dentro de un entorno local.

2.2. Objetivos Específicos

- Analizar el contexto operativo y tecnológico de la farmacia PharmaVictoria como base para el diseño de una arquitectura de software que responda a sus necesidades de gestión y control.

- Definir los requerimientos funcionales y no funcionales del sistema, estableciendo las características necesarias para garantizar seguridad, eficiencia y facilidad de mantenimiento.

- Aplicar patrones de diseño (GoF) que permitan estructurar una arquitectura de software con bajo acoplamiento, alta cohesión y capacidad de escalabilidad.

- Modelar la arquitectura del sistema utilizando el modelo C4, representando los niveles de contexto, contenedores, componentes y código para documentar la propuesta técnica de forma clara y ordenada.

- Evaluar y justificar las decisiones de diseño arquitectónico, explicando cómo los patrones aplicados y la estructura propuesta contribuyen a la optimización de los procesos de venta, inventario y administración de la farmacia.

III. Antecedentes bibliográficos

Antecedentes Nacionales:

1) ***"Desarrollar un sistema web para mejorar el control de inventario en la farmacia de la institución médica "Siempre Mujer" " (URP, 2021)***

Resumen:

La tesis desarrolla un sistema web para el control de inventario en la farmacia de la institución médica *Siempre Mujer*. Parte de un diagnóstico del manejo manual de los registros en papel y hojas de cálculo, proponiendo un sistema con módulos de movimientos entre almacenes, control diario de existencias y generación de reportes. Emplea una metodología de ingeniería de software que abarca las fases de análisis, diseño e implementación, logrando automatizar las operaciones de entradas, salidas, reportes y control de caducidades.

Aporte al proyecto:

- Presenta una estructura modular que puede orientar la organización de los componentes de *PharmaVictoria* (Inventario, Movimientos y Reportes).
- Demuestra resultados de eficiencia en la automatización del control de existencias y reportes, validando el impacto positivo de un sistema informático en farmacias locales.
- Su aplicación de fases metodológicas sirve como referencia para estructurar el ciclo de desarrollo y documentación del proyecto PharmaVictoria.

2) ***"Implementación de un sistema de información para la mejora de la gestión de la farmacia Megafarma" (UNCP, 2018)***

Resumen:

El estudio implementa un sistema informático para optimizar la gestión de la farmacia *Megafarma*, abarcando el registro de ventas, control de inventario, manejo de kardex y consultas de productos. A partir de un diagnóstico de procesos manuales, se desarrolla un sistema que automatiza los movimientos de entrada y salida de productos, genera kardex valorizado y emite reportes de apoyo a la toma de decisiones. La tesis incluye el diseño de la base de datos, pruebas funcionales y recomendaciones para la operación del sistema.

Aporte al proyecto:

- Proporciona un modelo de tratamiento del kardex y de valorización de inventarios útil para estructurar la base de datos y los cálculos de stock en *PharmaVictoria*.
- Aporta ejemplos de pruebas funcionales y criterios de aceptación aplicables a la validación del sistema propuesto.
- Ofrece aprendizajes sobre la implementación práctica de sistemas de gestión en farmacias locales, útiles para anticipar necesidades de adaptación y capacitación del personal.

Antecedentes Internacionales:

1) "*Development of Pharmacy Inventory Management System (PIMS) by implementing ABC analysis for Farmasi Hijrah*" (Universiti Teknologi MARA, Malasia, 2017)

Resumen:

Esta tesis desarrolla un sistema de gestión de inventario para la farmacia *Farmasi Hijrah*, con el objetivo de mejorar la clasificación de productos, el control de stock y la visibilidad del inventario. Identifica problemas como inventario excesivo, uso inadecuado de software, demoras para localizar productos y costos elevados asociados a productos sin rotación. Emplea la metodología Waterfall, desarrollando fases de identificación de requerimientos, análisis, diseño, desarrollo e implementación. El sistema plantea mejoras en el control de existencias, la optimización del inventario y la reducción de costos operativos.

Aporte al proyecto:

- Brinda una referencia sobre estrategias de organización de inventarios que pueden aplicarse en *PharmaVictoria* para mejorar la gestión de stock y la clasificación de productos.
- Aporta una estructura clara de requerimientos y diseño de base de datos útil para definir la gestión de productos y optimizar procesos de registro.
- Proporciona ejemplos de indicadores de eficiencia que pueden servir para evaluar el rendimiento del sistema una vez implementado.

2) ***Optimización e integración del sistema de gestión de inventarios FarmaBot para farmacias comunitarias en Chile"*** **(Universidad de Chile, 2024)**

Esta tesis presenta la optimización e integración del sistema *FarmaBot*, utilizado en farmacias comunitarias en Chile, con el propósito de modernizar la gestión del inventario y mejorar los procesos de dispensación de medicamentos. Parte de la problemática de sistemas obsoletos y procesos manuales que generaban errores en los registros y demoras en la atención. Se propone una solución tecnológica que integra nuevas funcionalidades de control de stock, seguimiento de vencimientos, generación automatizada de reportes y mejora en la interfaz de usuario. El proyecto demuestra una reducción significativa en los tiempos de operación y una mayor eficiencia en la trazabilidad de medicamentos.

Aporte al proyecto:

- Ofrece una referencia de modernización tecnológica que orienta cómo actualizar y optimizar sistemas de gestión farmacéutica.
- Presenta resultados de eficiencia en el control de vencimientos y generación automática de reportes, aplicables a los módulos de *PharmaVictoria*.
- Muestra un enfoque de adopción progresiva del sistema, útil para implementar *PharmaVictoria* sin afectar las operaciones diarias en farmacias locales.

IV. Marco Teórico

4.1. Arquitectura de Software

Definición:

La arquitectura de software es la estructura fundamental de un sistema informático, que define la organización de sus componentes, las relaciones entre ellos y los principios que guían su diseño y evolución. En esencia, establece la base técnica que soporta el desarrollo, mantenimiento y escalabilidad de una aplicación.

Importancia:

En el proyecto PharmaVictoria, la arquitectura de software tiene un rol esencial, ya que permite organizar el sistema de manera modular, garantizando su correcto funcionamiento y evolución. El sistema incluye los módulos de login, pantalla principal y los submódulos de punto de venta, inventario, clientes, proveedores, reportes, configuración y usuarios. Cada módulo cumple una función específica dentro del flujo operativo de la farmacia, manteniendo la integridad de los datos y la eficiencia del trabajo diario.

Una arquitectura bien definida permite que el sistema sea escalable, seguro, mantenible y flexible, de modo que pueda adaptarse fácilmente a nuevas funcionalidades, como facturación electrónica o sincronización.

Principios fundamentales:

- **Modularidad:**

El sistema se estructura en módulos independientes con funciones bien definidas, lo que permite una clara separación de responsabilidades. Por ejemplo, el módulo de punto de venta gestiona las transacciones, el de inventario administra productos y existencias, y el de reportes consolida la información para el análisis de ventas. Esta organización modular facilita el mantenimiento, la localización de errores y la ampliación del sistema sin afectar otras partes de la aplicación.

- **Seguridad:**

PharmaVictoria incorpora mecanismos que garantizan la integridad y protección de la información almacenada. Se implementa control de acceso por roles para restringir funciones según el tipo de usuario (administrador, vendedor), evitando modificaciones no autorizadas. Asimismo, se valida la información en cada capa del sistema y se manejan las conexiones a la base de datos de forma segura, reduciendo riesgos de filtraciones o pérdidas de datos.

- **Escalabilidad:**

El diseño de la aplicación considera su crecimiento futuro tanto en volumen de datos como en cantidad de usuarios. Gracias al uso del patrón Singleton junto con HikariCP, se optimiza el manejo de conexiones a la base de datos, permitiendo atender múltiples operaciones

de manera simultánea sin degradar el rendimiento. Esta característica asegura que el sistema pueda ampliarse o integrar nuevas funcionalidades sin requerir una reestructuración profunda.

En conjunto, la arquitectura de PharmaVictoria garantiza una solución sostenible, organizada y preparada para el crecimiento continuo de la empresa.

4.2. Patrones de Diseño (GoF)

Los patrones de diseño son soluciones reutilizables y probadas que resuelven problemas comunes en el desarrollo de software. Propuestos por el grupo *Gang of Four (GoF)*, se clasifican en tres tipos: creacionales, estructurales y de comportamiento.

En el proyecto PharmaVictoria, su aplicación es clave para lograr un diseño flexible, desacoplado y alineado con los principios de la arquitectura moderna.

A. Patrones Creacionales

Controlan la creación de objetos y su inicialización, otorgando flexibilidad y reduciendo el acoplamiento entre clases.

- **Singleton:**

  Utilizado en la clase DatabaseConfig, garantiza una única instancia del gestor de conexiones HikariCP, optimizando el acceso a la base de datos y evitando la creación innecesaria de múltiples conexiones. Se usa en los repositorios de los módulos de usuarios, ventas, inventario y auditoría.

- **Factory Method:**

  El patrón Factory Method permite crear objetos de distintos tipos, como comprobantes de venta, a través de una interfaz común. Así, se pueden instanciar boletas u otros comprobantes sin que el código cliente conozca las clases concretas, facilitando la extensión y el mantenimiento al agregar nuevos tipos de comprobantes o formatos en el futuro.

- **Builder:**

  El patrón Builder ya se aplica en la construcción de objetos complejos como Boleta, permitiendo asignar sus atributos de forma flexible y clara antes de crear la instancia final. Esto simplifica la creación de comprobantes y mejora la legibilidad y mantenibilidad del código, especialmente cuando se manejan múltiples parámetros.

B. Patrones Estructurales

Definen cómo se organizan y relacionan las clases y objetos del sistema, mejorando la flexibilidad y reduciendo la dependencia entre módulos.

- **Facade:**

  Aplicado a través de la clase ExportFacade, centraliza y simplifica la exportación de datos a Excel (Apache POI) y PDF (iText).

  De esta forma, los controladores de usuarios, clientes, inventario y proveedores invocan métodos como exportarUsuariosPDF() o exportarProveedoresExcel() sin conocer los detalles técnicos del proceso, reduciendo el acoplamiento y facilitando el mantenimiento.

C. Patrones de Comportamiento

Regulan la comunicación entre objetos, dividiendo claramente las responsabilidades y mejorando la coordinación interna del sistema.

- **Command:**

  Encapsula las operaciones de inventario (como ajustes de stock o movimientos de productos) dentro de objetos comando (InventarioCommand, AjusteStockCommand). Cada cambio se registra en la tabla de auditoría (audit_log), lo que proporciona trazabilidad completa, permite reconstruir el historial de modificaciones y facilita un posible sistema de "deshacer" (undo) en el futuro.

- **Observer:**

  Implementado mediante la clase SystemEventManager, notifica automáticamente a la interfaz gráfica cuando se producen cambios en la base de datos.

  Por ejemplo, al registrar una venta o actualizar un producto, se emiten eventos que los controladores de *Dashboard, Inventario o Clientes* reciben para actualizar las vistas en tiempo real sin intervención manual.

- **Strategy:**

  Permite cambiar dinámicamente los algoritmos de búsqueda y filtrado dentro de los módulos de *productos, clientes, proveedores y usuarios*.

  Cada criterio de búsqueda (por nombre, proveedor, laboratorio, vencimiento, etc.) se implementa como una clase independiente, de modo que se pueden agregar nuevos filtros sin modificar el controlador principal.

4.3. Modelado Arquitectónico

El modelado arquitectónico permite representar de forma visual y estructurada los distintos niveles de un sistema de software, facilitando la comprensión de sus componentes, relaciones e interacciones.

En el proyecto PharmaVictoria, se adopta el modelo C4, el cual descompone la arquitectura en cuatro niveles jerárquicos:

- **Contexto:** Muestra la relación del sistema con sus usuarios (administradores, vendedores) y con servicios externos como SUNAT o DIGEMID.
- **Contenedores:** Describe los principales elementos del sistema, como la aplicación de escritorio, la base de datos y los servicios auxiliares (exportación, auditoría, eventos).
- **Componentes:** Representa los módulos funcionales internos, como *punto de venta*, *inventario*, *clientes*, *proveedores*, *reportes*, *configuración* y *usuarios*.
- **Código:** Detalla la implementación a nivel de clases, donde se aplican los patrones de diseño que estructuran el comportamiento del sistema.

Este enfoque proporciona una visión clara, ordenada y escalable de la arquitectura, sirviendo como base para su análisis y documentación técnica posterior.

V. Desarrollo del proyecto

5.1. Descripción del Sistema

El sistema propuesto es una plataforma de gestión integral para la farmacia PharmaVictoria, desarrollada como una aplicación de escritorio en JavaFX con conexión a una base de datos MySQL, orientada a optimizar los procesos de venta, control de inventario, gestión de clientes, proveedores y usuarios.

El sistema cuenta con un módulo de acceso (login) y una pantalla principal desde la cual se puede navegar a los distintos módulos funcionales:

- **Punto de venta:** Permite registrar transacciones, emitir comprobantes y calcular totales.
- **Inventario:** Gestiona productos, lotes, precios, fechas de vencimiento y existencias en tiempo real.
- **Clientes:** Administra información de compradores frecuentes, historial de compras y puntos acumulados.
- **Proveedores:** Controla el registro, actualización y contacto de los proveedores de productos.
- **Reportes:** Permite generar y exportar informes de ventas en formatos PDF y Excel, utilizando estrategias y fábricas para seleccionar el tipo y formato de reporte. Facilita la consulta, visualización y análisis de la información relevante del sistema.

- **Configuración:** Permite ajustar parámetros clave del sistema, como notificaciones por correo electrónico, permisos de vendedor, respaldos y restauración de la base de datos, y alertas de vencimiento de productos.
- **Usuarios:** Gestiona roles, accesos y credenciales del personal autorizado.

El objetivo general del sistema es digitalizar y centralizar la gestión operativa de la farmacia, mejorando la eficiencia en las ventas, la trazabilidad del inventario y la precisión en la información registrada. Además, se busca fortalecer la seguridad de los datos y optimizar el flujo de trabajo diario.

5.2. Requerimientos

Los requerimientos del sistema definen las funciones y condiciones bajo las cuales operará la aplicación *PharmaVictoria*. Los requerimientos funcionales describen las acciones que el sistema debe realizar para cumplir con sus objetivos operativos, como la gestión de ventas, inventario, usuarios y reportes. Por su parte, los requerimientos no funcionales establecen las características de calidad que garantizan el correcto desempeño de dichas funciones, como la seguridad, disponibilidad, rendimiento y mantenibilidad. En conjunto, ambos tipos de requerimientos aseguran que el sistema no solo cumpla sus funciones principales, sino que también lo haga de manera eficiente, segura y confiable.

Requerimientos Funcionales:

ID	Requerimiento Funcional	Descripción
RF-01	Registro de usuarios y roles	El sistema permite registrar, editar y eliminar usuarios, asignándoles roles como administrador o vendedor.
RF-02	Inicio de sesión y control de acceso	El sistema permite el inicio de sesión según el rol del usuario y restringe las funciones disponibles de acuerdo con sus permisos.
RF-03	Gestión de productos	El sistema permite registrar, modificar, eliminar y consultar productos con sus detalles (nombre, stock, precio y fecha de vencimiento).
RF-04	Control de inventario	El sistema registra entradas, salidas y ajustes de stock en tiempo real.
RF-05	Registro de ventas	El sistema permite registrar ventas en mostrador y genera boletas o facturas electrónicas según corresponda.
RF-06	Gestión de proveedores	El sistema almacena la información de los proveedores y sus productos asociados.
RF-07	Reportes administrativos	El sistema genera reportes de ventas, niveles de stock, productos próximos a vencer y movimientos por período.
RF-08	Auditoría de acciones	El sistema registra todas las operaciones realizadas por cada usuario para garantizar el control interno y la trazabilidad.
RF-09	Copia de seguridad de datos	El sistema permite realizar y restaurar copias de seguridad locales de la base de datos.

Requerimientos No Funcionales:

ID	Requerimiento No Funcional	Descripción	Relacionado con
RNF-01	Seguridad de acceso	Las credenciales de los usuarios se almacenan cifradas y se validan antes de permitir el acceso al sistema.	RF-01, RF-02
RNF-02	Rendimiento	Cada operación de registro o consulta se completa en menos de 3 segundos bajo condiciones de carga normal.	RF-03, RF-04, RF-05
RNF-03	Usabilidad	La interfaz es clara, consistente y adaptable a distintos tamaños de pantalla en equipos de escritorio.	Todos los RF
RNF-04	Disponibilidad	El sistema está disponible al menos el 99% del tiempo durante el horario laboral.	RF-04, RF-05, RF-07
RNF-05	Escalabilidad	La aplicación permite la conexión simultánea de varios usuarios sin degradar el rendimiento.	RF-04, RF-05
RNF-06	Integridad de datos	Las operaciones de actualización o eliminación confirman transacciones antes de aplicarse en la base de datos.	RF-03, RF-04, RF-08
RNF-07	Recuperación ante fallos	Existe un mecanismo de respaldo y restauración ante pérdida o corrupción de datos.	RF-09
RNF-08	Mantenibilidad	El código está estructurado por capas (presentación, lógica y datos), facilitando futuras modificaciones.	Todos los RF
RNF-09	Confidencialidad	La información sensible de clientes, ventas y usuarios no se muestra sin los permisos de acceso correspondientes.	RF-02, RF-05, RF-07

5.3. Patrones de Diseños Aplicados

En el desarrollo del sistema PharmaVictoria, se aplicaron diversos patrones de diseño del catálogo GoF, con el objetivo de mejorar la estructura interna del código, reducir el acoplamiento entre módulos y facilitar la mantenibilidad del sistema.

Estos patrones se integran en los principales componentes de la aplicación, fortaleciendo la arquitectura modular y en capas (MVC).

El sistema implementa de manera activa 7 patrones (Singleton, Facade, Command, Observer, Factory Method, Builder y Strategy).

A continuación, se resumen los patrones aplicados en el proyecto y su función dentro de la arquitectura del sistema:

Patrón	Tipo	Descripción	Justificación
Singleton	Creacional	En el proyecto, el patrón **Singleton** se aplica en la clase ***DatabaseConfig*** para garantizar una única instancia encargada de manejar la conexión a la base de datos. Esto centraliza la configuración del pool de conexiones y permite administrar los recursos de forma más eficiente en toda la aplicación.	El uso del patrón Singleton en DatabaseConfig está justificado porque asegura que toda la aplicación utilice una única configuración y pool de conexiones a la base de datos, evitando inconsistencias, duplicidad de recursos y facilitando la gestión centralizada y eficiente de las conexiones.
Factory Method	Creacional	En el proyecto, el patrón **Factory Method** se aplica en ***ComprobanteFactory*** para centralizar la creación de distintos tipos de comprobantes. Esto permite generar el tipo adecuado según la solicitud y facilita agregar nuevos comprobantes sin modificar la lógica existente.	El uso de este patrón permite crear comprobantes de manera flexible y centralizada, facilitando la incorporación de nuevos tipos sin modificar el código cliente. Así, se promueve la escalabilidad y el mantenimiento del sistema, ya que la lógica de instanciación queda desacoplada del resto de la aplicación.

Patrón	Tipo	Descripción	Justificación
Builder	Creacional	El patrón **Builder** se usa en ***BoletaBuilder*** para construir boletas de manera flexible y ordenada, asignando sus atributos paso a paso antes de generar la instancia final. Esto facilita manejar comprobantes con varios parámetros y mejora la claridad del código.	El patrón Builder se emplea para facilitar la creación de objetos Boleta con múltiples atributos, permitiendo construir instancias de manera clara y controlada. Esto reduce errores, mejora la legibilidad y hace más sencillo modificar o extender la construcción de comprobantes en el futuro.
Facade	Estructural	En el proyecto, la clase ***ExportFacade*** usa el patrón **Facade** para ofrecer una interfaz unificada que gestiona la exportación de datos en distintos formatos. Esto simplifica procesos complejos y permite que el sistema interactúe con la exportación de manera más fácil y centralizada.	El uso del patrón Facade en **ExportFacade** es adecuado porque permite centralizar y simplificar la exportación de datos, ocultando la complejidad de los procesos internos y facilitando que otras partes del sistema realicen exportaciones de manera sencilla, coherente y sin duplicar lógica.
Command	Comportamieno	El patrón **Command** se aplica mediante la clase abstracta ***InventarioCommand*****,** que encapsula operaciones del inventario y permite ejecutarlas, deshacerlas y registrarlas de forma flexible y desacoplada. Esto facilita la gestión de acciones y el control de su historial.	El patrón **Command** encapsula cada operación del inventario como un objeto independiente, permitiendo ejecutarla, deshacerla y registrarla con flexibilidad. Esto mantiene un historial de movimientos y desacopla la lógica de las acciones, mejorando el control y la mantenibilidad del sistema.
Observer	Comportamieno	El patrón **Observer** se implementa mediante la interfaz ***SystemEventObserver***, permitiendo que varios componentes se suscriban y reaccionen automáticamente a eventos del sistema. Esto favorece una comunicación desacoplada, donde los observadores reciben notificaciones y actúan ante cambios sin depender directamente del emisor.	El patrón **Observer** permite que distintas partes del sistema reaccionen automáticamente a eventos sin acoplarse entre sí, mejorando la escalabilidad y el mantenimiento. Además, facilita añadir o modificar observadores sin afectar el resto de la aplicación.
Strategy	Comportamieno	El patrón **Strategy** se aplica en clases como ***SearchByVencimientoStrategy*****,** permitiendo definir y cambiar dinámicamente distintos algoritmos de búsqueda en el inventario. Esto facilita extender y	El patrón Strategy permite definir distintas formas de búsqueda en el inventario sin modificar el código principal, facilitando su extensión y mantenimiento. Esto permite agregar nuevas estrategias de

Patrón	Tipo	Descripción	Justificación
		reutilizar diferentes estrategias de filtrado sin modificar el código principal.	manera flexible y desacoplada según las necesidades del sistema.

VI. Modelado Arquitectónico

6.1. Modelo C4

El modelo C4 permite visualizar la arquitectura del sistema PharmaVictoria en distintos niveles de abstracción, desde la interacción general con los usuarios hasta la implementación a nivel de código. A continuación, se presenta una versión simplificada adaptada al contexto del proyecto.

Nivel 1 – Contexto

El sistema PharmaVictoria es una aplicación de escritorio desarrollada para optimizar la gestión integral de una farmacia local.
Permite a los distintos actores del entorno: administradores, personal de mostrador y proveedores, realizar tareas relacionadas con las ventas, el control de inventario, el abastecimiento de productos, la emisión de boletas y la generación de reportes administrativos. Asimismo, el sistema se comunica con entidades externas como SUNAT (para la facturación electrónica) y DIGEMID (para la validación de medicamentos), cumpliendo así con los requisitos normativos del sector farmacéutico.

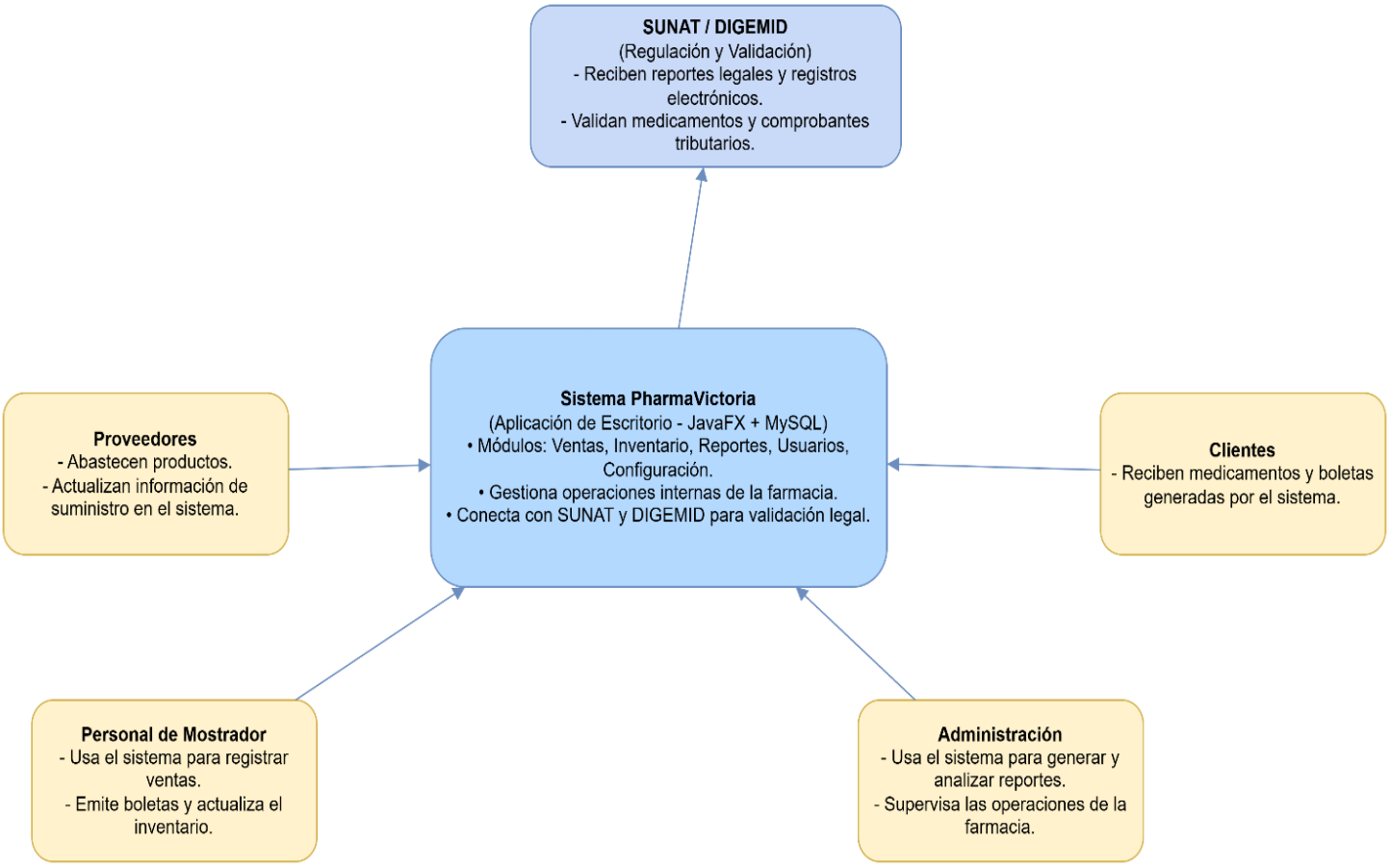
Elementos:

- **Actores humanos:**

- ***Clientes:*** *reciben medicamentos y boletas generadas por el sistema en el punto de venta.*
- ***Personal de mostrador:*** *utiliza el sistema para registrar ventas, actualizar el inventario y emitir comprobantes.*
- ***Administración:*** *emplea el sistema para acceder a reportes, supervisar operaciones y analizar indicadores de gestión.*
- ***Proveedores:*** *abastecen productos y actualizan la información de suministro mediante los módulos de inventario y compras.*

- **Sistema central:**
  - ***Sistema PharmaVictoria (Aplicación de escritorio):*** *Plataforma desarrollada en JavaFX conectada a una base de datos MySQL, que integra los módulos de ventas, inventario, clientes, proveedores, usuarios, reportes y configuración.*
- **Sistemas externos:**
  - *SUNAT → encargado de la facturación electrónica y validación tributaria.*
  - *DIGEMID → sistema regulador del Ministerio de Salud para la validación de medicamentos y control sanitario.*

DIAGRAMA DE CONTEXTO – Nivel 1 (C4)



Nivel 2 – Contenedores

El sistema PharmaVictoria se compone de varios contenedores que trabajan de manera integrada para permitir la gestión completa de una farmacia local.

Cada contenedor cumple una función específica dentro del entorno técnico y operativo del sistema, facilitando la separación de responsabilidades, la mantenibilidad y la escalabilidad de la aplicación.

Descripción general:

La aplicación PharmaVictoria se ejecuta en un entorno de escritorio y se conecta con una base de datos local que almacena la información de productos, ventas, usuarios y proveedores. El sistema interactúa con un módulo de facturación electrónica encargado de comunicar los comprobantes a SUNAT, mientras que un servidor de respaldo se encarga de mantener copias de seguridad periódicas de la base de datos.
Finalmente, todas las computadoras están interconectadas mediante una red local, permitiendo la sincronización y el acceso compartido a la información.

Elementos del modelo de contenedores:

Aplicación de Escritorio (JavaFX)

- Contiene la interfaz de usuario y la lógica de negocio.
- Gestiona los módulos de ventas, inventario, reportes, usuarios y proveedores.

- Se comunica con la base de datos mediante JDBC y el pool de conexiones HikariCP.
- Implementa los patrones de diseño Facade, Command, Observer y Strategy.

Base de Datos Local (MySQL)

- Almacena la información estructurada de productos, clientes, proveedores, ventas, usuarios y auditorías.
- Garantiza la integridad de datos, la disponibilidad local y la seguridad mediante respaldos.
- Permite consultas concurrentes mediante el uso del patrón Singleton en la conexión.

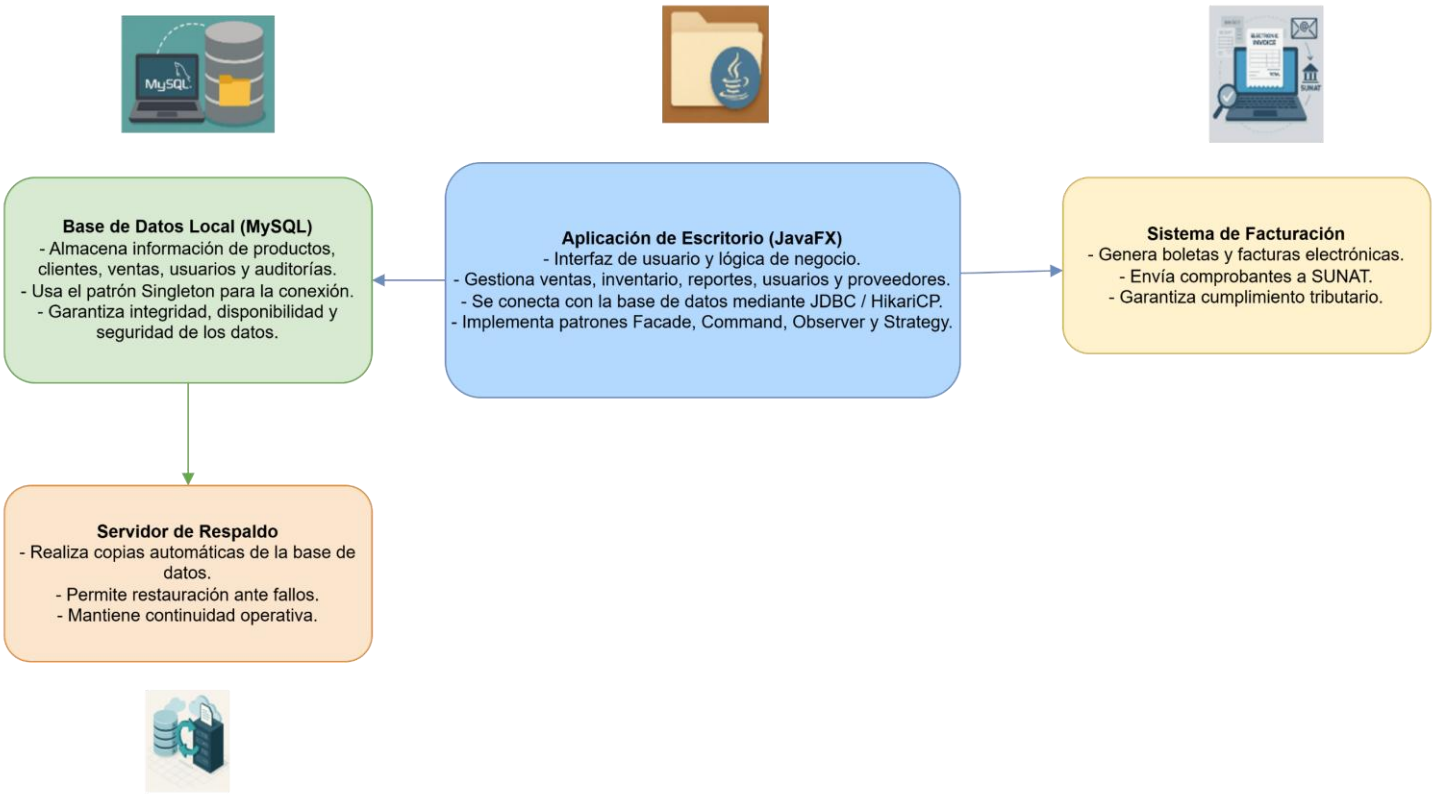
Sistema de Facturación

- Genera comprobantes electrónicos (boletas y facturas).
- Comunica la información de ventas directamente a SUNAT mediante API o archivos XML.
- Asegura el cumplimiento normativo y tributario de las operaciones.

Servidor de Respaldo

- Ejecuta copias automáticas de la base de datos local a intervalos definidos.
- Permite restaurar la información ante fallos o pérdidas de datos.

DIAGRAMA DE CONTENEDORES – Nivel 2 (C4)



Nivel 3 - Componentes (Backend)

El Nivel 3 representa la estructura interna de la Aplicación de Escritorio, mostrando los módulos funcionales y los componentes (controladores, clases y servicios) que componen el sistema *PharmaVictoria*.

Cada componente cumple un rol específico dentro del flujo operativo de la farmacia, garantizando integridad de datos, seguridad y escalabilidad mediante la aplicación de patrones de diseño GoF.

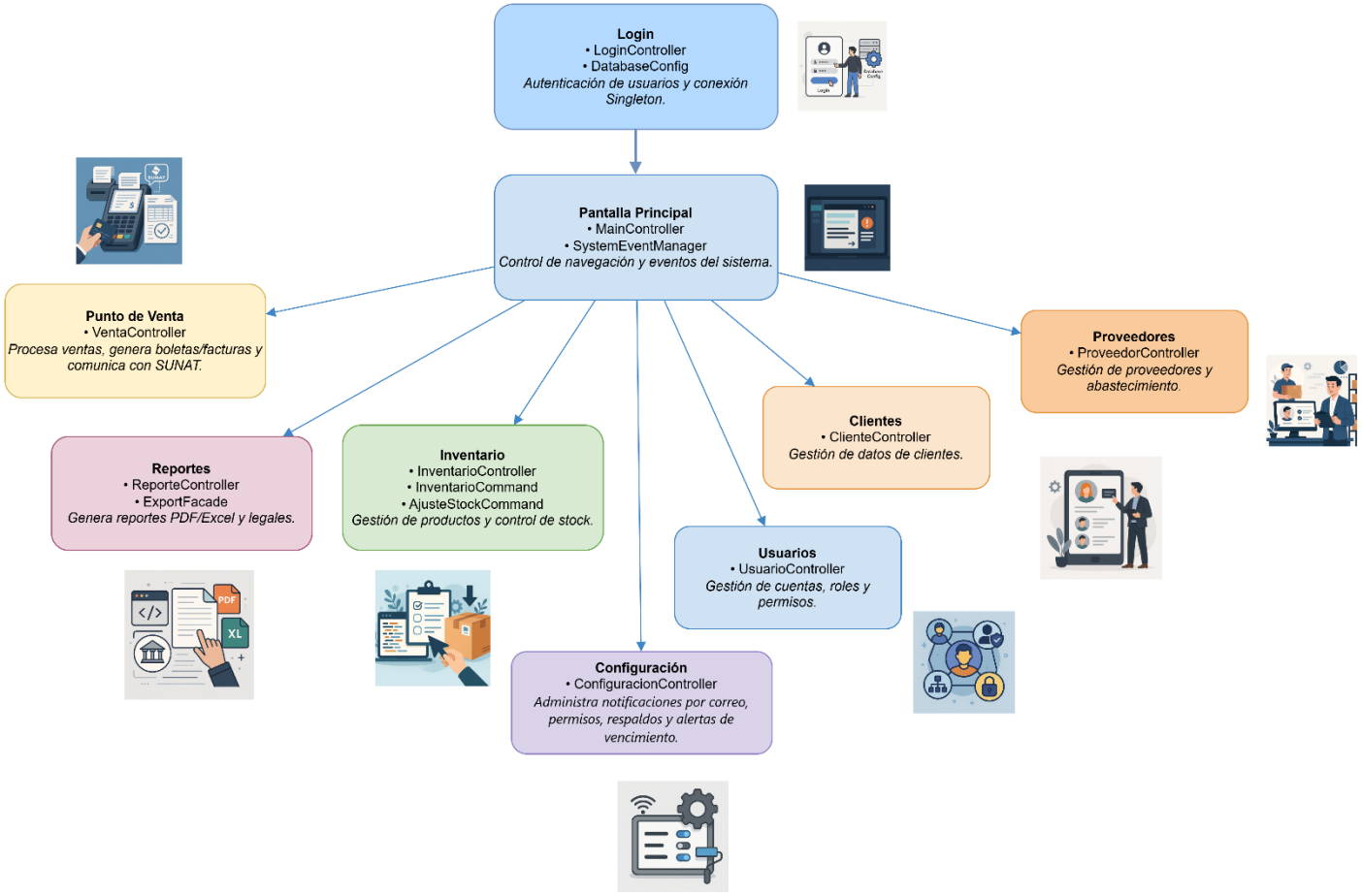
Estructura general del sistema

La aplicación está dividida en módulos principales y submódulos funcionales, cada uno controlado por componentes backend (clases y controladores):

Módulo / Submódulo	Componentes principales (Clases / Controladores)	Funciones principales	Patrones o características
Login	LoginController, DatabaseConfig	Autentica usuarios, valida credenciales y roles (administrador / vendedor). Redirige a la pantalla principal tras inicio de sesión.	Singleton (conexión a DB), MVC
Pantalla Principal	MainController, SystemEventManager	Controla la navegación entre submódulos, administra ventanas activas y actualiza la interfaz en tiempo real.	Observer (eventos del sistema), MVC

Punto de Venta	VentaController	Registra ventas, calcula totales, emite boletas/facturas y comunica datos al sistema de facturación (SUNAT).	Observer, MVC
Inventario	InventarioController, InventarioCommand, AjusteStockCommand	Administra productos, lotes, precios y vencimientos. Ajusta stock y registra movimientos.	Command, MVC
Clientes	ClienteController	Registra, busca y edita información de clientes. Vincula datos con el módulo de ventas.	MVC, validación de datos
Proveedores	ProveedorController	Registra, edita y consulta información de proveedores. Actualiza datos de abastecimiento.	MVC
Reportes	ReporteController, ExportFacade	Genera reportes de ventas por período, por producto y por usuario	Facade, MVC
Configuración	ConfiguracionController	Administra notificaciones por correo, permisos de vendedor, respaldos y restauración de base de datos, y alertas de vencimiento de productos.	Configuración dinámica
Usuarios	UsuarioController	Gestiona cuentas, roles, permisos y control de acceso del personal.	Control de roles, seguridad interna

DIAGRAMA DE COMPONENTES – Nivel 3 (C4)



Nivel 4 – Código

El Nivel 4 muestra la implementación real del sistema PharmaVictoria, desarrollado en JavaFX con conexión a MySQL.

En este nivel se representan las clases más importantes que materializan la arquitectura lógica definida en los niveles anteriores.

Se agrupan en tres bloques principales: Controladores, Servicios y Repositorios/Configuración, reflejando el flujo interno de datos desde la interfaz de usuario hasta la base de datos.

Grupo	Clases más importantes	Responsabilidad principal
Controladores (UI)	LoginController, DashboardController, ProductosController	Gestionan la interfaz gráfica y los eventos del usuario. Se comunican con los servicios para procesar datos.
Servicios (Lógica de Negocio)	UsuarioService, ProductoService, ProveedorService	Aplican las reglas de negocio, validan datos y coordinan las operaciones entre los controladores y los repositorios.
Repositorios y Configuración (Acceso a Datos)	UsuarioRepository, ProductoRepository, DatabaseConfig	Acceden directamente a la base de datos MySQL mediante consultas SQL, gestionando la persistencia de la información.

Fragmentos representativos:

LoginController.java

```
@FXML
private void handleLogin() {
    logger.info("Iniciando proceso de autenticación");
    // Validar campos de usuario y contraseña
    if (!validateFields()) {
        return;
    }
    // Deshabilita la interfaz mientras se autentica
    setUIEnabled(false);
    // Crea una tarea en segundo plano para autenticar al usuario
    Task<Usuario> loginTask = new Task<>() {
        @Override
        protected Usuario call() throws Exception {
            String username = usernameField.getText().trim();
```

```
        String password = passwordField.getText();
        logger.debug("Intentando autenticar usuario: {}", username);
        // Llama al servicio de autenticación
        return authService.authenticate(username, password);
    }
};
// Si la autenticación es exitosa
loginTask.setOnSucceeded(e -> {
    Usuario usuario = loginTask.getValue();
    if (usuario != null) {
        handleSuccessfulLogin(usuario); // Navega al dashboard
    } else {
        handleFailedLogin("Credenciales incorrectas");
    }
});
// Si ocurre un error durante la autenticación
loginTask.setOnFailed(e -> {
    Throwable exception = loginTask.getException();
```

```
        logger.error("Error durante autenticación: {}", exception.getMessage(), exception);
        handleFailedLogin("Error interno del sistema");
    });
    // Ejecuta la tarea en un hilo separado para no bloquear la interfaz
    new Thread(loginTask).start();
}
```

UsuarioService.java

```
// Registrar usuario en el sistema
public String registrarUsuario(Usuario usuario, Long adminId) {
    // Validaciones: username único, email único, DNI único
    if (usuarioRepository.existsByUsername(usuario.getUsername())) {
        return "Ya existe un usuario con ese nombre de usuario.";
    }
    if (usuario.getEmail() != null && !usuario.getEmail().isEmpty()) {
        if (usuarioRepository.existsByEmail(usuario.getEmail())) {
            return "Ya existe un usuario con ese email.";
        }
    }
```

```
if (usuario.getDni() != null && !usuario.getDni().isEmpty()) {
    if (usuarioRepository.existsByDni(usuario.getDni())) {
        return "Ya existe un usuario con ese DNI.";
    }
}
// Guarda el usuario en la base de datos
Usuario guardado = usuarioRepository.save(usuario);
if (guardado == null) {
    return "No se pudo guardar el usuario en la base de datos.";
}
// Registra el cambio en el historial de auditoría
UsuarioHistorialCambio cambio = new UsuarioHistorialCambio();
cambio.setUsuario_id((int)guardado.getId().longValue());
cambio.setCampo_modificado("CREACION");
cambio.setValor_anterior(null);
cambio.setValor_nuevo("Usuario creado");
cambio.setModificado_por(adminId != null ? adminId.intValue() : null);
cambio.setFecha(java.time.LocalDateTime.now());
```

```
        historialCambioRepository.save(cambio);
        return null;
    }
```

UsuarioRepository.java

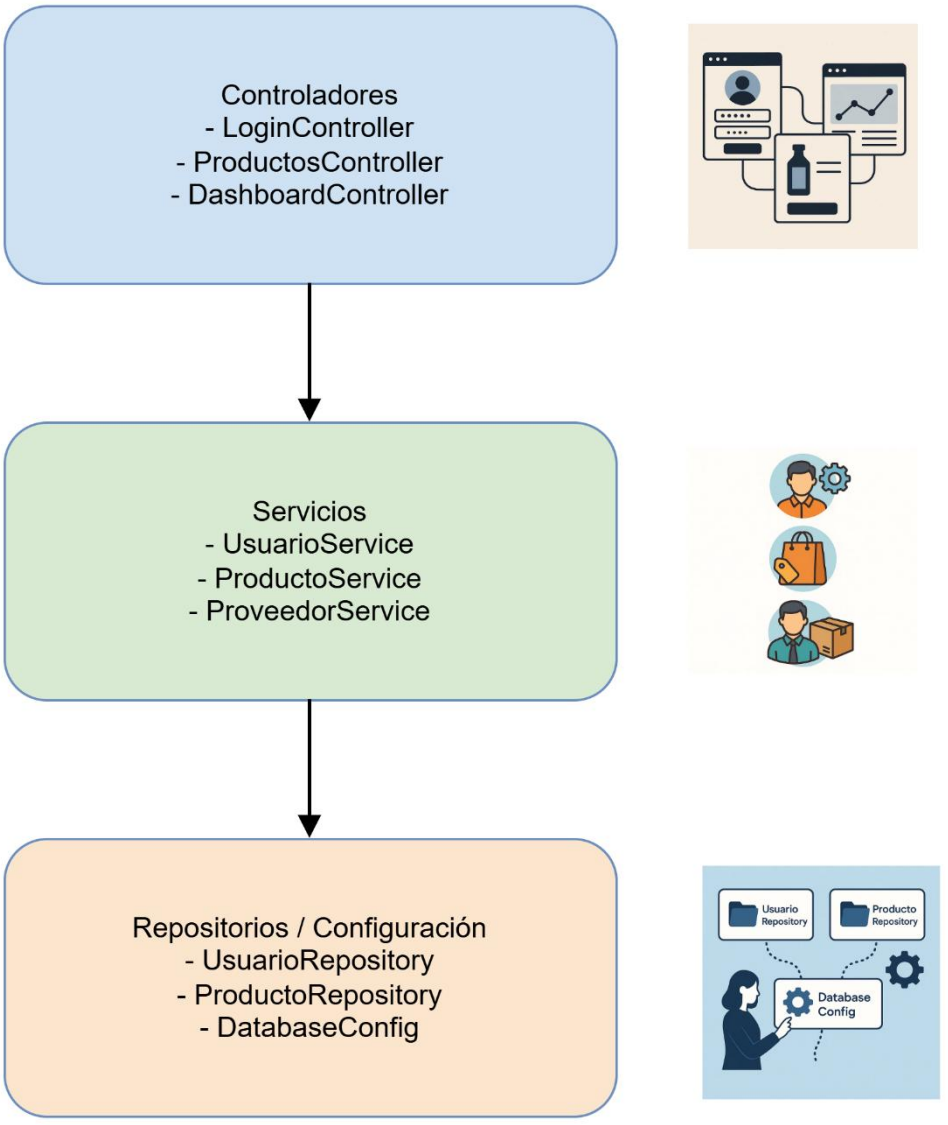
```
    /**
     * Verifica si existe un usuario con el email dado
     */
    public boolean existsByEmail(String email) {
        String sql = "SELECT COUNT(*) FROM usuarios WHERE email = ?";
        try (Connection conn = databaseConfig.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, email);
            try (ResultSet rs = stmt.executeQuery()) {
                if (rs.next()) {
                    return rs.getInt(1) > 0; // Devuelve true si existe al menos un usuario con ese
email
                }
            }
        } catch (SQLException e) {
```

```
        logger.error("Error al verificar existencia de email: {}", email, e);
    }
    return false;
}
```

DatabaseConfig.java

```
// Singleton para configuración y pool de conexiones a la base de datos
public static synchronized DatabaseConfig getInstance() {
    if (instance == null) {
        instance = new DatabaseConfig();
    }
    return instance;
}
// Obtiene una conexión lista para usar
public Connection getConnection() throws SQLException {
    return dataSource.getConnection();
}
```

DIAGRAMA DE CÓDIGO – Nivel 4 (C4)



VII. Resultados Esperados

El desarrollo del sistema PharmaVictoria busca obtener una solución de software sólida, escalable y documentada, que sirva como base para la gestión integral de una farmacia.
De acuerdo con los avances e implementaciones actuales, se esperan los siguientes resultados concretos:

• Arquitectura modular documentada

Se espera contar con una arquitectura modular y en capas (MVC), debidamente documentada mediante el modelo C4, que refleje la estructura lógica, física y de componentes del sistema.
Cada módulo: punto de venta, inventario, clientes, proveedores, reportes, configuración y usuarios, se encuentra separado y definido, permitiendo la ampliación y mantenimiento independiente de cada parte.

• Implementación funcional de componentes clave

Hasta el momento, se ha logrado implementar exitosamente los módulos principales del sistema, los cuales permiten realizar las operaciones esenciales de la farmacia:

- **Login y control de usuarios:** Acceso seguro con roles diferenciados (administrador y vendedor).
- **Punto de venta:** Registro de transacciones, cálculo de totales y actualización automática del inventario.

- **Inventario:** Gestión completa de productos, lotes, precios y auditoría de movimientos.
- **Clientes y proveedores:** Administración de registros y búsqueda filtrada.
- **Reportes:** Exportación de datos en PDF y Excel a través del patrón *Facade*.

Estos módulos interactúan correctamente con la base de datos MySQL y el sistema de conexión optimizado con HikariCP, garantizando rendimiento y estabilidad.

• Diagramas arquitectónicos bien estructurados

El proyecto incluye diagramas basados en el modelo C4, que representan los distintos niveles de abstracción del sistema (contexto, contenedores, componentes y código).
Este diagrama refleja la organización interna del sistema y la comunicación entre módulos, facilitando la comprensión de su estructura técnica.

• Justificación técnica de los patrones aplicados

El sistema aplica correctamente cinco patrones de diseño principales (*Singleton, Facade, Command, Observer* y *Strategy*), los cuales mejoran la mantenibilidad, reactividad, seguridad y trazabilidad del sistema.
Adicionalmente, se prevé incorporar los patrones Factory Method y Builder en futuras versiones, para mejorar la creación y configuración de objetos complejos.

VIII. Conclusiones

La aplicación de los patrones de diseño GoF y del modelo arquitectónico C4 en el desarrollo del sistema PharmaVictoria permitió establecer una arquitectura modular, segura y escalable, cumpliendo con los objetivos de optimizar la gestión de ventas, inventario, facturación y reportes administrativos de la farmacia.

Los patrones de diseño contribuyeron a mejorar la organización interna del código, reducir el acoplamiento entre módulos y facilitar la mantenibilidad, permitiendo un control eficiente de recursos, la actualización reactiva de la interfaz y la incorporación de nuevas funcionalidades sin afectar la estabilidad del sistema.

El modelo C4 permitió representar visualmente la arquitectura en diferentes niveles, desde el contexto general hasta los componentes y detalles de implementación, facilitando la comprensión de las interacciones entre los elementos del software y los actores internos y externos, incluyendo la integración con servicios regulatorios como SUNAT y DIGEMID. Esto asegura una documentación clara y ordenada, útil para futuras ampliaciones o mantenimientos.

En conjunto, la solución obtenida es robusta, flexible y alineada con las necesidades operativas y tecnológicas de la farmacia, demostrando cómo una arquitectura de software bien diseñada contribuye a modernizar y optimizar los procesos internos. Además, se contempla la futura incorporación de los patrones Builder y Factory Method, con el fin de mejorar la creación de objetos complejos y reforzar la modularidad y escalabilidad del sistema.

IX. Bibliografía

Azahar, N. (2017). *Development of Pharmacy Inventory Management System (PIMS) by implementing ABC analysis for Farmasi Hijrah* (Tesis de grado). Universiti Teknologi MARA, Faculty of Computer and Mathematical Sciences, Jasin Campus, Melaka. https://ir.uitm.edu.my/id/eprint/18795

Cisneros Delao, I. M. (2019). *Implementación de un sistema de información para la mejora de la gestión de la farmacia Megafarma – Lima, 2018* (Tesis de pregrado, Universidad Nacional del Centro del Perú). Repositorio Institucional UNCP. https://hdl.handle.net/20.500.12894/5306

Palma Canelo, A. (2024).*Optimización e integración del sistema de gestión de inventarios FarmaBot para farmacias comunitarias en Chile.* (Tesis de grado). Universidad de Chile. https://repositorio.uchile.cl/handle/2250/205275

Vilchez Lecca, G y Vilchez Lecca, J. (2021). *Desarrollar un sistema web para mejorar el control de inventario en la farmacia de la institución médica para la mujer: "Siempre Mujer".* Universidad Ricardo Palma.

https://repositorio.urp.edu.pe/entities/publication/d33921ee-1053-4137-9fe6-b6b9e5c00e61